

Module 1: Lesson 4

Hyperparameter Optimization on Keras Tuner



Outline

- ▶ Regression problem: Testing the Keras tuner
- ▶ Keras Tuner
- ▶ Timing AAPL stock using tuned model hyperparameters

Testing the Keras Tuner

In this practical lesson, we'll revisit the problem of timing AAPL stock (or any other stock/financial time series for that matter) that we faced in lesson 2 of this module. This time, however, we'll make it a regression (rather than classification) problem:

- ▶ To do this, we will use an MLP framework in a regression problem to predict the future 25-day of AAPL stock
- ▶ Different from previous examples, here, we will incorporate hyper-parameter tuning via the Keras Tuner.
- ▶ This whole exercise yields two main takeaways:
 - (i) Hyperparameter tuning is **computationally costly** (even in our simple case, it takes up to 5 minutes of computations).
 - (ii) Hyperparameter tuning can have a **substantial effect** on the predictive performance of our models (even simple tuning), especially in these first initial stages for us.
- ▶ For the complete documentation on the Keras tuner, please visit: https://keras.io/api/keras_tuner/

Keras Tuner

The HPO task is much facilitated by the Keras tuner: a library for hyperparameter tuning in TensorFlow.

- ▶ To properly use it, we first need to define our **hypermodel**: which parameters should we tune? what choices are available?
- ▶ In this case, we will tune: (i) # hidden layers, (ii) # units in hidden layers, and (iii) dropout rate:

```
class MLP_model(HyperModel):
    def build(self, hp):

        # We define a constant activation function of ReLU form. We will not be tuning the activation functions
        act_fun = 'relu'

        # We do ask the Keras Tuner to choose whether is best to have a dropout rate after each hidden layer of 0.2 or 0.3
        n_dropout = hp.Choice('n_dropout', values=[0.20, 0.30])

        model = tf.keras.models.Sequential()

        # Now, we will use a loop to let the tuner choose the number of layers that is best for the model between 1 and 5
        for i in range(1, hp.Int("num_layers", 1, 5)):
            # Within this loop, we will also ask the tuner to decide the optimal number of units that each of the selected layer should have.
            model.add(tf.keras.layers.Dense(units= hp.Int("units_dense_" + str(i), min_value=1, max_value=25, step=5), activation=act_fun))
            model.add( tf.keras.layers.Dropout(n_dropout))

        model.add(tf.keras.layers.Dense(units = 1))

        # As was the case with the activation function, there is no tuning on the learning rate, nor the optimizer, although we could
        # easily incorporate it.
        hp_lr = 1e-5
        adam = tf.keras.optimizers.Adam(learning_rate = hp_lr)
        model.compile(optimizer = adam, loss='mean_absolute_error')

        return model
```

Keras Tuner

Once we have built our hypermodel, it's time to put it to work:

- Our choice of optimizer (i.e., tuner) will be the **hyperband**:

Li, Lisha, et al. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization." *The Journal of Machine Learning Research*, vol. 18, no. 1, 2017, pp. 6765–6816, <https://www.jmlr.org/papers/volume18/16-558/16-558.pdf>

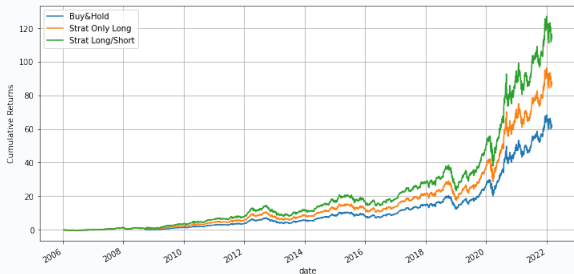
- Hyperband requires:
 - Hypermodel: just defined!
 - Objective: how to optimize? → Minimum loss in validation
 - Other parameters: setting random seed, storage of results, etc.

```
hypermodel=MLP_model()
tuner = kt.Hyperband(hypermodel,
                    objective = kt.Objective("val_loss", direction="min"),
                    overwrite = True,
                    max_epochs = 30,
                    seed = 1234,
                    directory=os.path.normpath('C:/'))

# Let's run the tuner! (Warning: this could take time)
tuner.search(X_train, y_train, validation_split=val_split)
best_hps = tuner.get_best_hyperparameters(num_trials = 1)[0]
```

Trading strategy based on tuned model predictions

Once we have tuned our model hyperparameters, we can check how a trading strategy based on model predictions in the test sample would have performed via **backtesting**:



One more time, please do remember the caveats associated with backtesting:

- It only validates the performance of the strategy for the test sample—no external validity from this!

Summary of Lesson 4

In Lesson 4, we have looked at:

- ▶ Keras tuner for hyperparameter optimization
- ▶ Performance of Keras tuner in a regression problem

⇒ **References for this lesson:**

Yu, Tong, and Hong Zhu. "Hyperparameter Optimization: A Review of Algorithms and Applications." *arXiv*, 2020, <https://arxiv.org/abs/2003.05689>

Introduction to Keras tuner example in TensorFlow:

https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en

TO DO NEXT: Go to the associated **Jupyter Notebook** to cover the full content of this lesson.

In the next module, we'll continue to dig deeper into the Deep Learning world by moving forward to more advanced and complex models and techniques.